

1. 背景与目标

1.1 现状

模型平台访问路径：

https://model.dev.beemwk.com:8805/login （https://10.224.231.142:8805/login）

https://model-platform.beemchat.com/

管理员账户：

admin@beem.com， 密码123456

测试账号：

test@beem.com 111111

代码仓：

https://gitlab.beemwk.com/beem-ai/model-platform （需要打开一下网页，出了404后，管理员才能加入进来）

当前模型平台处于 Demo 阶段，基本具备：

-
- 多模型代理转发（通过 LiteLLM）
-
- AppKey 管理（创建/撤销，含预算上限和月度 Token 限额）
-
- 用量计费统计（按 Token/音频时长/字符数计费）
-
- 组织/成员/配额管理
-
- 管理后台和审计日志（管理动作级别）

1.2 投产前的核心问题

问题

影响

无请求级别日志

出现问题无法定位，无法统计各业务实际吞吐

无监控告警

模型服务异常无感知，无法主动运维

限速策略固定（60 RPM 全局硬编码）

无法区分不同业务场景的优先级

无并发/降级机制

高并发时会议等高优业务与 RAG、试用用户争抢资源

AppKey 粒度的策略配置缺失

无法对不同 AppKey 施加差异化的频控和优先级策略

1.3 目标

本轮改造分为两个方向：

1. 日志与监控体系：建立完整的请求级日志，接入指标监控和告警，为运维提供可观测性基础。
2. 分级调用策略体系：在 AppKey 管理上增加每个 AppKey 对每个模型的调用策略配置，实现优先级保障与降级控制，仅管理员可操作。

1.4 说明

1. 后端服务当前用的python， 可以根据实际情况更改技术框架。但是litellm是一个做好的python版本模型转发框架，若要换，需要评估成本
2. 核心需求其实就是Key的角色， 权限管理， 流量管控

2. 需求一：日志与监控体系

2.1 请求访问日志

每一次对 /v1/chat/completions、/v1/audio/*、/v1/embeddings 的请求，都需要记录一条结构化日志：

字段

类型

说明	
trace_id	
string	
全链路追踪 ID，由 Backend 生成，透传到 LiteLLM	
request_id	
string	
单次请求 UUID	
timestamp	
datetime	
请求到达时间（UTC）	
appkey_id	
int	
调用方 AppKey 内部 ID	
appkey_prefix	
string	
AppKey 显示前缀（如 sk-xxx...xxx），便于快速识别	
org_id	
int	
所属组织	
model_id	
string	
请求模型名（如 qwen3.5-35b-a3b）	
endpoint	
string	
/v1/chat/completions、/v1/audio/transcriptions 等	
http_method	
string	
POST	
status_code	
int	
HTTP 响应码	
error_code	
string null	
业务错误码（如 RATE_LIMITED、QUOTA_EXCEEDED、BUDGET_EXCEEDED）	
latency_ms	
int	
从收到请求到返回第一个字节的时间（TTFB，流式场景）	
total_latency_ms	
int	
从收到请求到完整响应结束的时间	
prompt_tokens	
int	
输入 Token 数	
completion_tokens	
int	
输出 Token 数	
audio_duration_seconds	
float null	
音频时长（ASR 场景）	
cost_coins	
float	
本次请求费用（单位：coins）	

client_ip
string
调用方 IP
user_agent
string
调用方 UA
upstream_latency_ms
int
Backend 调用 LiteLLM 的耗时（排除 Backend 自身处理）
upstream_status
int
LiteLLM 返回的 HTTP 状态码
priority_tier
string
本次请求所属优先级（见第 4 节，如 P0/P1/P2）
queue_wait_ms
int
在限速队列等待的时长（非排队则为 0）
is_streaming
bool
是否为流式请求
服务部署在k8s, 日志会被ES采集

2.2 Prometheus 指标

在 Backend中增加 /metrics 端点（使用 prometheus-client 库），暴露以下指标：

2.2.1 请求指标

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14

请求总数（按 model、appkey、status_code、priority_tier 打标签）

model_platform_requests_total{model, appkey_prefix, status_code, priority_tier, endpoint}

请求延迟分布（Histogram，P50/P95/P99）

model_platform_request_latency_seconds{model, endpoint, priority_tier}

当前活跃请求数

model_platform_active_requests{model, priority_tier}

限速拒绝次数

model_platform_rate_limited_total{model, appkey_prefix, reason}

队列等待延迟

model_platform_queue_wait_seconds{model, priority_tier}

2.2.2 Token/用量指标

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

Token 消耗（Counter）

model_platform_tokens_total{model, token_type} # token_type: prompt | completion

音频时长消耗

model_platform_audio_seconds_total{model}

费用（按 org、model）

model_platform_cost_coins_total{org_id, model}

2.2.3 系统健康指标

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

上游（LiteLLM/vLLM）错误率

model_platform_upstream_errors_total{model, status_code}

上游响应延迟

model_platform_upstream_latency_seconds{model}

AppKey 当前 RPM 使用率（Gauge）

model_platform_appkey_rpm_usage{appkey_prefix, model2}

2.3 Grafana 仪表盘（需运维配置）

文档提供以下仪表盘模板需求，由运维侧在 Grafana 中实现：

- 1. 全局概览：总 QPS、P95 延迟、错误率、Token 消耗趋势（按小时/天）
- 2. 模型维度：每个模型的 QPS、延迟、错误分布、Token 消耗
- 3. AppKey 维度：Top N AppKey 的用量、费用、限速触发次数
- 4.

优先级分析：各优先级（P0/P1/P2）的流量占比、等待时长、降级触发次数

2.4 告警规则（需运维配置）

告警

触发条件

严重级别

高错误率

任意模型 5 分钟内 5xx 错误率 > 5%

Critical

P95 延迟飙升

chat completions P95 > 30s（持续 3 分钟）

Warning

上游不可达

上游连续 3 次返回 5xx

Critical

P0 请求被限速

P0 优先级 RATE_LIMITED 出现

Critical

配额耗尽

任意 org 月度配额使用率 > 90%

Warning

2.5 控制台日志查询 API

新增以下 API（仅管理员可访问完整数据，普通用户仅可查看自己 AppKey 的数据）：

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

GET /api/admin/request-logs

参数：start_time, end_time, appkey_id, model_id, status_code, error_code, priority_tier

分页：limit (max 200), offset

返回：request_logs 列表

GET /api/admin/request-logs/stats

参数：start_time, end_time, group_by (model|appkey|org|hour)

返回：QPS、平均延迟、错误率、Token 汇总的聚合统计

GET /api/request-logs ← 普通用户接口，仅返回当前 AppKey 的数据

3. 需求二：分级调用策略体系

3.1 策略模型设计

3.1.1 三级优先级

级别

名称

典型场景

保障策略

P0

关键

在线会议 ASR+翻译+总结

优先调度，不降级，RPM 不受全局并发限制

P1

重要

办公 Chatbot、IM 消息翻译

正常调度，高负载时轻微限速， 比如等待XX秒以内

P2

普通/试用

RAG、试用用户 Chatbot

高负载时主动限速或排队，可降级

3.1.2 每个 AppKey × 每个模型的策略配置

允许管理员为每个 AppKey 配置其在不同模型上的调用策略

3.1.3 全局模型并发配置

由管理员配置每个模型的全局并发上限和 PO 保障容量

大致模式：

3.1.4 计费逻辑

不通模型有不通的计费方式

•

chat模型，按照input token， output token 不通价格来计费，一般output token要贵一些（缓存命中的费用往后放一放）

•

embedding模型

•

text embedding， 以input token 计费

•

image embedding， 以图像张数计费

•

ASR

•

流式调用， 以连接建立和断开中间的总时长计费

•

文件调用， 以音频文件时长计费

•

TTS， 以输入的文字长度计费

•

图片生成，以生成的张数计费（目前没有，占位）

•

视频生成， 以生成的视频时长计费， 若有参考图，多一张图，额外单加一定费用（目前只有sora2， 以占位为主）

整体计费， 以【低消+用量计费】的逻辑为主。

3.2 策略执行流程

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

Request → Backend /v1/chat/completions

↓

1. AppKey 认证 & 状态检查 (现有)
2. 查询 appkey_model_policies (appkey_id × model_id, fallback 到 '*')
3. 读取 priority_tier、rpm_limit、max_concurrency 等

↓

4. [限速检查] Redis 计数器, key = rate:{appkey_id}:{model_id}:{minute}
超过 rpm_limit → 返回 429, error_code = RATE_LIMITED

↓

5. [并发检查] Redis Gauge, key = concurrency:{model_id}

读取 model_capacity_config.max_global_concurrency

if 当前并发 >= max_global_concurrency:

if priority_tier == 'P0':

if 当前并发 < max_global_concurrency + p0_reserved_concurrency:

继续 (P0 占用保障容量)

else:

返回 503 (极端情况, P0 也超容)

elif priority_tier == 'P1':

按 degradation_action 处理 (通常是 queue, 等待有空位)

elif priority_tier == 'P2':

按 degradation_action 处理 (queue / reject)

elif 当前并发 >= overload_threshold:

P2 请求进入排队或拒绝

P0/P1 继续

↓

6. 并发计数 +1 (Redis INCR, 请求完成后 DECR)

↓

7. 预算/配额检查 (现有)

↓

8. 转发 LiteLLM (携带 trace_id)

↓

9. 并发计数 -1, 写请求日志 (异步)

3.3 策略管理 API

所有接口仅系统管理员 (is_system_admin = True) 可调用。

3.3.1 AppKey 策略配置

key对应优先级的设置

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31
- 32
- 33
- 34
- 35

查看某 AppKey 的所有策略

GET /api/admin/appkeys/{appkey_id}/policies

Response: [

```
{
  "model_id": "",
  "priority_tier": "P2",
  "rpm_limit": 30,
  "max_concurrency": 5,
  "degradation_action": "queue",
  "queue_timeout_seconds": 30
},
{
  "model_id": "qwen3.5-35b-a3b",
  "priority_tier": "P1",
```

```
"rpm_limit": 60,
...
}
]
```

创建或更新某 AppKey 对某模型的策略

```
PUT /api/admin/appkeys/{appkey_id}/policies/{model_id}

Body: {
  "priority_tier": "P0", // 必填
  "rpm_limit": 200, // null = 不限
  "tpm_limit": null,
  "max_concurrency": 20,
  "queue_timeout_seconds": 30,
  "max_queue_size": 100,
  "degradation_action": "none", // P0 不降级
  "daily_token_limit": null,
  "daily_request_limit": null
}
```

删除策略（恢复为全局默认）

```
DELETE /api/admin/appkeys/{appkey_id}/policies/{model_id}
```

3.3.2 模型容量配置

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20

查看所有模型的容量配置

```
GET /api/admin/model-capacity

Response: [
{
  "model_id": "qwen3.5-35b-a3b",
  "max_global_concurrency": 50,
  "p0_reserved_concurrency": 10,
```

```
"p1_max_concurrency": 30,
"overload_threshold": 40
}
]
```

更新某模型的容量配置

```
PUT /api/admin/model-capacity/{model_id}

Body: {
  "max_global_concurrency": 50,
  "p0_reserved_concurrency": 10,
  "p1_max_concurrency": 30,
  "overload_threshold": 40
}
```

3.3.3 实时状态查询

```
1
2
3
4
5
6
7
8
9
10
11
12
```

查看当前各模型并发状态（实时，读 Redis）

```
GET /api/admin/model-capacity/status

Response: [
{
  "model_id": "qwen3.5-35b-a3b",
  "current_concurrency": 23,
  "max_global_concurrency": 50,
  "overload_threshold": 40,
  "p2_queue_length": 0,
  "is_overloaded": false
}
]
```

3.4 429 响应格式规范

当触发限速或降级时，统一返回以下格式（便于业务方处理）：

```
1
2
3
4
5
6
7
8
9
10
```

HTTP 429 Too Many Requests

```
{
  "error": {
    "code": "RATE_LIMITED", // 或 CONCURRENCY_EXCEEDED / QUEUE_TIMEOUT / QUEUE_FULL
    "message": "Rate limit exceeded for this AppKey on model qwen3.5-35b-a3b",
    "retry_after": 15, // 建议重试等待秒数（可选）
    "limit_type": "rpm", // rpm | concurrency | daily_request | daily_token
    "current_limit": 60,
    "priority_tier": "P2"
  }
}
```

3.4 初始化建议

部署时，管理员应先完成以下配置再开放业务接入：

1.

在 model_capacity_config 表中为每个模型录入并发上限（根据 GPU 资源评估）

2.

为会议服务的 AppKey 设置 P0 策略（priority_tier=P0, degradation_action=none）

3.

为试用用户的 AppKey 设置 P2 策略（rpm_limit=20, degradation_action=reject）

4.

保持 RAG 服务 AppKey 默认 P2, degradation_action=queue, queue_timeout_seconds=60

4. 验收标准

验收项

标准

请求日志完整性

每次 chat/audio 请求均有对应 request_logs 记录，字段无缺失

P0 保障

模拟 vLLM 并发打满（concurrency=max_global_concurrency）时，P0 请求成功率 $\geq 99\%$

P2 降级

并发超过 overload_threshold 时，P2 请求按策略排队或返回 429

RPM 限速准确性

设置 RPM=60 的 AppKey，在 1 分钟内第 61 次请求起返回 429

指标可观测

/metrics 端点可被 Prometheus 抓取，核心指标数值符合预期

策略 API 鉴权

非系统管理员调用策略管理 API 返回 403

审计日志

策略的创建/修改/删除操作在 audit_logs 中有记录

TODO：

•

重构后端，google-stt 隐藏（已经隐藏，重构的话不用做）

•

监控体系

•

key和model的管理（技术实现上需要细聊）

5. 相关资源

开发环境：

tidb: 10.96.2.179: 4000 账号: root 密码 BeeM@AIdB

redis:

host: r-2zeypx5d7hxy4qh9t.redis.rds.aliyuncs.com

port: 6379

pwd: 63U8T_uFAVt0u

测试环境

tidb:

nlb-0sz05o4ilrolbwscz.me-central-1.nlb.aliyuncsslbintl.com: 4000

账号beemai

密码U2V0dvToWpRd

redis:

host: r-l4v12wpb8u7yffkbw3.redis.me-central-1.rds.aliyuncs.com

port: 6379

pwd: sYX7552zYkfE

生产环境

tidb:

nlb-12ug9nz72avcfmdlq4.me-central-1.nlb.aliyuncsslbintl.com 端口 4000

账号 beemai

密码 FezNemi7Xupy

redis:

host: r-l4vv5nbljkre3yfh59.redis.me-central-1.rds.aliyuncs.com

port: 6379

pwd: 862Q96WuCpjdinzc

6.

已有模型

模型汇总:

模型名字

模型用处

模型url

Jina-Embedding

对文本/图像做embedding处理

http://jina-embedding-v4.test.beemai.svc

GPT-OSS-120B

文本对话

http://openai-gpt.test.beemai.svc/v1

ASR-file

对音频文件做转录

http://qwen3-asr-vllm.test.beemai.svc

ASR-stream

流式转录音频

ws://bm-ai-meet-asr-translation.test.beemai.svc/ws/translate

Habibi-tts

文本转语音

http://10.224.231.142:9800

都是测试环境

访问需要配置/etc/hosts:

10.224.19.122 openai-gpt.test.beemai.svc

10.224.19.122 bm-ai-translation.test.beemai.svc

10.224.19.122 bm-ai-asr-translation.test.beemai.svc

10.224.19.122 bm-ai-im-asr-translation.test.beemai.svc

10.224.19.122 qwen3-asr-vllm.test.beemai.svc

10.224.19.122 jina-embedding-v4.test.beemai.svc

•

embedding模型:

http://jina-embedding-v4.test.beemai.svc

•

文本：

1

```
curl -X POST "http://jina-embedding-v4.test.beemai.svc/v1/embeddings" -H "Content-Type: application/json" -d '{"model": "jina-embedding-v4", "input": "Hello world"}'
```

返回结果：取prompt tokens， 作为文本的计费标准

1

```
{"object": "list", "data": [{"object": "embedding", "embedding": [-0.0036646330263465643, -0.027926696464419365, 547009, ...], "index": 0}], "model": "jina-embedding-v4", "usage": {"prompt_tokens": 2, "total_tokens": 2, "image_count": 0, "text_count": 1}}
```

•

图像（可以是url格式，也可以是base 64）

1

```
curl -s http://jina-embedding-v4.test.beemai.svc/v1/embeddings -H "Content-Type: application/json" -d '{"model": "jina-embedding-v4", "input": "https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x92dp.png"}'
```

返回结果：取image_count作为计费标准

1

```
{"object": "list", "data": [{"object": "embedding", "embedding": [0.03017108514904976, -0.020992625504732132, ...], "index": 0}], "model": "jina-embedding-v4", "usage": {"prompt_tokens": 0, "total_tokens": 0, "image_count": 1, "text_count": 0}}
```

•

混合文本和图像输入

1

2

3

```
curl -s http://jina-embedding-v4.test.beemai.svc/v1/embeddings \
-H "Content-Type: application/json" \
-d '{"model": "jina-embedding-v4", "input": ["Hello world", "https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x92dp.png"]}'
```

返回结果：既有prompt tokens， 也有image_count

1

```
{"object": "list", "data": [{"object": "embedding", "embedding": [-0.0036646330263465643, ...], "index": 1}], "model": "jina-embedding-v4", "usage": {"prompt_tokens": 2, "total_tokens": 2, "image_count": 1, "text_count": 1}}
```

Image Base64的使用：

1

2

3

4

5

6

7

8

9

```
curl -s http://jina-embedding-v4.test.beemai.svc/v1/embeddings \
-H "Content-Type: application/json" \
-d '{
  "model": "jina-embedding-v4",
  "input": [
    "Hello world",
    ""
  ]
}'
```

•

chat 模型：

http://openai-gpt.test.beemai.svc/v1

1

2
3
4
5
6
7
8
9
10
11
12

```
curl -X POST http://openai-gpt.test.beemai.svc/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <YOUR_API_KEY>" \
-d '{
  "model": "GPTOssModel",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello, who are you?"}
  ],
  "temperature": 0.7,
  "max_tokens": 1000
}'
```

output: 红色标出了真实回复和token计数（当前用的gpt-oss有chat template， 所以input token数量每次多50token左右）

1
2
3
4
5
6
7
8
9
10
11

```
{
  "id": "chatcmpl-c75453d89a53288cf238ee80cae88433",
  "object": "chat.completion",
  "created": 1781182920,
  "model": "GPTOssModel",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! I'm ChatGPT, an AI language model created by OpenAI. I'm here to help answer your questions, brainstorm ideas, explain concepts, or just have a friendly chat—whatever you need. How can I assist you today?",
        "refusal": null,
        "annotations": null,
        "audio": null,
        "function_call": null,
        "tool_calls": [],
        "reasoning": "We need to respond as ChatGPT, friendly. Probably introduce self.",
        "reasoning_content": "We need to respond as ChatGPT, friendly. Probably introduce self.",
        "logprobs": null,
        "finish_reason": "stop",
        "stop_reason": null,
        "token_ids": null
      },
      "service_tier": null,
      "system_fingerprint": null,
      "usage": {
        "prompt_tokens": 81,
        "total_tokens": 154,
        "completion_tokens": 73,
        "prompt_tokens_details": null,
        "prompt_logprobs": null,
        "prompt_token_ids": null,
        "kv_transfer_params": null
      }
    }
  ]
}
```

•
ASR
•

非流式，文件调用

<http://qwen3-asr-vllm.test.beemai.svc> （统一用下方的地址， 包含了音频前处理， 此地址需要设定）

1
2
3
4
curl -X POST http://qwen3-asr-vllm.test.beemai.svc/v1/audio/transcriptions \
-F "file=@/Users/songyuhao/Downloads/raw_input.wav" \
-F "response_format=json" \
-F "enable_denoise=true" -F "enable_vad=true"
ws://bm-ai-meet-asr-translation.test.beemai.svc/ws/translate
1
ws://bm-ai-meet-asr-translation.test.beemai.svc:80/ws/translate?asrDst=auto&fragmentNotify=1
.

chat 模型 - gemma4 （此模型还未加入模型平台）
http://gemma4-vllm.test.beemai.svc

1
2
3
4
5
6
7
8
curl -X POST http://gemma4-vllm.test.beemai.svc/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
"model": "text-model-small",
"messages": [
{"role": "user", "content": "Hello"}
]
}'
.

TTS模型

服务地址： http://10.224.231.142:9800

1
2
3
curl -X POST http://10.224.231.142:9801/v1/audio/speech \
-H 'Content-Type: application/json' \
-d '{"input": "مرحبا", "voice": "role_01", "speed": 1.3}' -o fast.wav # 1.3 倍速
可用角色列表： GET http://10.224.231.142:9801/v1/audio/speech/voices